# SOFTWARE ENGINEERING REVISION GUIDE

**Complete Syllabus Coverage (Units 1 to 5) — 1-Hour Exam Preparation**

**FAST REVISION STRATEGY**

**How to use this guide for 1-Hour Revision:** Spend **10 minutes per Unit** reviewing key definitions, process models, and diagrams. Spend the final **10 minutes** on the *Quick Revision Formula Sheet* and testing your recall with the *20 Exam Practice Questions*.

## UNIT 1: INTRODUCTION TO SOFTWARE & SOFTWARE ENGINEERING

### 1.1 Software Fundamentals & Characteristics

**Software Definition:** A collection of instructions, data, or programs used to operate computers and perform specific tasks. It is non-tangible, serving as both a **product** (delivers computing capability) and a **vehicle** (delivers control and information processing).

#### Key Characteristics of Software:

• **Software is Developed or Engineered:** Unlike physical goods manufactured in factories, software requires creative problem-solving, analysis, design, coding, and testing.

• **Software Doesn't Wear Out:** It is not susceptible to environmental degradation. However, it deteriorates over time due to frequent changes/maintenance introduce side effects (spikes in failure rate curve).

• **Custom-Built vs Component-Based:** Modern software is increasingly assembled using reusable software components rather than built entirely from scratch.

### 1.2 Attributes of Good Software & Application Domains

**Attributes of Good Software:** Correctness, Reliability, Efficiency, Usability, Maintainability, Portability, Scalability, Security, Reusability, and Interoperability.

**Software Application Domains:** System Software (OS, drivers), Application Software (payroll, banking), Engineering/Scientific (MATLAB, AutoCAD), Embedded (microwave, ABS braking), Product-line (inventory packages), Web/Mobile Apps (browser/mobile platforms), and AI Software (expert systems, neural networks, ChatGPT).

### 1.3 Software Engineering as Layered Technology

**Software Engineering (IEEE):** The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.

| Layer | Focus & Description |
|---|---|
| Quality Focus | The bedrock foundation (TQM, Six Sigma) driving organizational culture and continuous improvement. |
| Process | The glue connecting technologies; defines framework activities, milestones, quality assurance, and change control. |
| Methods | Provides technical 'how-to' for tasks: requirements analysis, design modeling, coding, testing, and support. |
| Tools | Automated or semi-automated support for processes and methods (CASE: Computer-Aided Software Engineering). |

### 1.4 Software Development Myths

| Category | Myth | Reality |
|---|---|---|
| Management | Standards books are all we need. | Standards are useless unless enforced, practiced, and kept up to date. |
| Management | Adding people speeds up late projects. | Brook's Law: Adding manpower to a late project makes it later due to communication overhead. |
| Customer | Requirements can easily change later. | Late changes multiply costs exponentially due to rework in design, code, and tests. |
| Developer | Job is done when code works. | 60-80% of all software effort occurs AFTER delivery (maintenance and support). |

| Category | Myth | Reality |
|---|---|---|
| Developer | Quality can only be judged when running. | Formal technical reviews (FTR) catch defects far earlier and cheaper than testing. |

## 1.5 Software Process Framework & Umbrella Activities

### 5 Generic Process Framework Activities:

**1. Communication:** Requirements gathering and stakeholder collaboration.

**2. Planning:** Resource estimation, scheduling, tracking, and risk analysis.

**3. Modeling:** Creating architectural blueprints, analysis models, and UI designs.

**4. Construction:** Code generation and verification testing.

**5. Deployment:** Delivery to customer, feedback collection, and maintenance.

**Umbrella Activities (Run across entire project):** Software Project Tracking & Control, Risk Management, Software Quality Assurance (SQA), Technical Reviews, Measurement, Software Configuration Management (SCM), Reusability Management, and Work Product Preparation.

## 1.6 Software Process Models

| Model | Core Characteristics | Best Used When... |
|---|---|---|
| Waterfall | Linear sequential lifecycle. High discipline, rigid phases. | Requirements are well-understood, stable, and unvarying. |
| Incremental | Delivers releases in usable chunks (increments). Reduces initial delivery time. | Core functionality needed quickly; detailed features refined later. |
| Prototyping | Iterative build-and-evaluate cycles using mockups. Clarifies fuzzy needs. | Customer cannot define detailed requirements upfront. |
| Spiral | Risk-driven iterative loops combining prototyping with waterfall rigor. | Large-scale, high-risk, mission-critical complex projects. |
| RAD | Rapid Application Development using component-based construction. | Tight deadlines (60-90 days), well-known scope, highly modular. |

## 1.7 Agile Software Development & Scrum

**Agile Manifesto Values:** (1) **Individuals & Interactions** over processes and tools; (2) **Working Software** over comprehensive documentation; (3) **Customer Collaboration** over contract negotiation; (4) **Responding to Change** over following a plan.

**Scrum Framework:** Agile process centered on short, timeboxed cycles called **Sprints** (1-4 weeks). Key elements include *Product Backlog* (prioritized requirements), *Sprint Backlog*, and daily 15-minute *Standup Meetings* led by a **Scrum Master** covering 3 questions: What did you do yesterday? What will you do today? What obstacles exist?

# UNIT 2: SOFTWARE REQUIREMENT ANALYSIS AND SPECIFICATION

## 2.1 Requirement Elicitation & Analysis

**Requirement Engineering:** The set of tasks and methods used to establish what a system should do. **Stakeholders:** Anyone affected by or having interest in the system.

**Elicitation Techniques:** Interviews (one-on-one), Focus Groups / Workshops, Questionnaires/Surveys, Direct Observation, Context Diagrams, and Benchmarking / Competitive Analysis.

## 2.2 Functional vs Non-Functional Requirements (NFRs)

| Attribute | Functional Requirements (FRs) | Non-Functional Requirements (NFRs) |
|---|---|---|
| Definition | Specifies WHAT the system must do (functions/behaviors). | Specifies HOW WELL the system performs (quality attributes). |
| Focus | Specific user features, inputs, outputs, calculations. | System properties, operational constraints, performance. |
| Examples | User login, generate PDF invoice, search catalogue. | 99.95% availability, < 2 sec response time, AES-256 encryption. |

**Key NFR Categories:** Availability (uptime), Reliability (bug-free execution), Response Time (latency), Scalability (workload expansion), Security (protection), Maintainability, Usability, Recoverability, Throughput.

### 2.3 Requirement Engineering 7 Phases

**1. Inception:** Establish initial business need and broad project scope.
**2. Elicitation:** Gather requirements from stakeholders using various techniques.
**3. Elaboration:** Refine requirements into detailed functional and behavioral models.
**4. Negotiation:** Resolve conflicting stakeholder priorities based on cost and risk.
**5. Specification:** Document requirements in a standard format (e.g., SRS).
**6. Validation:** Formally review and audit SRS for accuracy, completeness, and consistency.
**7. Management:** Track and control changes throughout the software lifecycle.

### 2.4 Software Requirements Specification (SRS) & IEEE Structure

**IEEE 830 SRS Structure:** 1. Introduction (Purpose, Scope, Definitions), 2. Overall Description (Product Perspective, User Classes, Constraints), 3. Specific System Requirements (Functional, External Interfaces: User/Hardware/Software/Comm, NFRs), 4. Appendices & Glossary.

**Characteristics of a Good SRS:** Correct, Complete, Unambiguous, Consistent, Verifiable, Modifiable, Traceable.

### 2.5 Requirement Modeling & UML Diagrams

| Diagram Type | Primary Purpose | Key Notation & Elements |
|---|---|---|
| Use Case | Shows system interactions from user perspective. | Actors, Use Cases (ovals), System Boundary, <>, <>. |
| Activity | Flow of control/operations (like a flowchart). | Activities (rounded rect), Decision diamonds, Fork/Join bars, Initial/Final. |
| Swimlane | Partitions activity diagrams by organizational role/dept. | Vertical/Horizontal swimlanes assigning responsibility. |
| Sequence | Shows interaction over time between objects. | Lifelines (dashed), Activation boxes, Synchronous/Asynchronous messages. |
| State | Models dynamic behavior of an object based on events. | States (rounded box), Event transitions [guard conditions], Initial/Final. |
| Class | Static structural model of classes and relationships. | Class compartments (Name, Attributes, Operations), Visibility (+,-,#,~). |
| DFD | Flow of data through functions and stores. | Entities (rect), Processes (circle/oval), Data Stores (open rect), Flows. |

## UNIT 3: SOFTWARE DESIGN

### 3.1 Core Design Concepts & Principles

• **Abstraction:** Hiding lower-level details to focus on high-level concepts (Data & Procedural abstraction).
• **Modularity:** Dividing software into separately named and addressable components.
• **Information Hiding:** Concealing internal implementation details within a module behind a crisp interface.
• **Encapsulation:** Bundling data attributes and operations together into a single unit (class).
• **Refinement:** Top-down process of elaborating high-level abstractions into detailed logic.

### 3.2 Software Design Pyramid & FURPS Quality Model

**Software Design Pyramid Layers:** Data/Class Design → Architectural Design → Interface Design → Component-Level Design.

**FURPS Model (HP):** **F**unctionality (features, capabilities), **U**sability (human factors, aesthetics), **R**eliability (failure frequency, recoverability), **P**erformance (speed, throughput, resource usage), **S**upportability (extensibility, maintainability).

### 3.3 Cohesion and Coupling

| Metric | Definition | Goal | Types / Levels |
|---|---|---|---|
| Cohesion | Measures internal strength/focus within a single module. | HIGH Cohesion (Ideal) | Functional (best), Sequential, Communicational, Procedural, Temporal, Logical, Coincidental (worst). |
| Coupling | Measures degree of interdependence between different modules. | LOW Coupling (Ideal) | Data (best), Stamp, Control, External, Common, Content (worst). |

### 3.4 Architectural & User Interface Design

**3-Tier Architecture:** (1) *Presentation Layer* (UI), (2) *Business Logic Layer* (Services/Rules), (3) *Data Access Layer* (DB).

### 3 Golden Rules for UI Design (Mandel):

**1. Place User in Control:** Provide flexible, undoable, customizable interactions without forcing unwanted actions.

**2. Reduce User's Memory Load:** Use meaningful defaults, intuitive shortcuts, visible cues, and real-world metaphors.

**3. Make Interface Consistent:** Maintain uniform layout, visual cues, and operating rules across all screens.

**Web App Design Pyramid:** Interface → Aesthetic → Content → Navigation → Architecture → Component Design.

# UNIT 4: SOFTWARE CODING AND TESTING

## 4.1 Coding Standards & Guidelines

**Coding Standards:** Rules covering variable naming (camelCase/UPPERCASE), global variable limits, standardized file headers, and exception handling. Enhances consistency, readability, maintainability, and team collaboration.

**Coding Guidelines:** Keep functions concise (e.g., < 10-20 lines), avoid `goto`, avoid deep nesting, single purpose identifiers, and mandatory self-documenting comments.

## 4.2 Code Reviews & Documentation

| Type | Formality | Process / Participants | Primary Goal |
|---|---|---|---|
| Walkthrough | Informal | Author guides peers through code; dry-run with test cases. | Discover algorithmic/logical errors. |
| Inspection | Formal | 3-8 members (Author, Inspector, Reader, Recorder). Structured checklist. | Catch classical programming errors & standards compliance. |

**Documentation Types:** *Internal Documentation* (comments, variable names, function headers inside code) vs. *External Documentation* (SRS, Design docs, User manuals, Installation guides).

## 4.3 Verification vs Validation & Testing Levels

| Aspect | Verification | Validation |
|---|---|---|
| Question | 'Are we building the product RIGHT?' | 'Are we building the RIGHT product?' |
| Nature | Static checking (reviews, walkthroughs, inspections). No code execution. | Dynamic testing (executing code against user requirements). |
| Responsibility | Quality Assurance (QA) team. | Testing Team & End Users. |

### 4 Progressive Levels of Testing:

**1. Unit Testing:** Tests smallest isolated units (classes/methods). Uses **Drivers** (dummy main for bottom-up) and **Stubs** (dummy subprogram for top-down).

**2. Integration Testing:** Tests interfaces between combined units. Strategies: Big Bang, Top-Down, Bottom-Up.

**3. System Testing:** Tests fully integrated application as a whole against system specs.

**4. Validation / Acceptance Testing:** Ensures compliance with business requirements. Includes **Alpha Testing** (at developer site) and **Beta Testing** (at live customer sites).

## 4.4 Testing Methodologies: White Box vs Black Box

| Dimension | White-Box Testing (Structural / Glass-Box) | Black-Box Testing (Behavioral / Functional) |
|---|---|---|
| Focus | Internal program logic, code structure, loops, branches. | External functionality, inputs, outputs, user interface. |
| Knowledge | Requires full knowledge of source code and internal logic. | Requires NO knowledge of code; based purely on SRS. |
| Techniques | Basis Path Testing, Cyclomatic Complexity, Loop Testing. | Boundary Value Analysis (BVA), Equivalence Partitioning. |

### Specialized Testing Types:

**Regression Testing:** Re-running old test suites after code changes to ensure no new defects were introduced.

**Smoke Testing:** Daily build stability check targeting critical pathways before detailed testing.

**System Testing Types:** Recovery Testing (fault tolerance), Security Testing (penetration resistance), Stress Testing (abnormal loads), Performance Testing (speed/throughput), Deployment Testing (multi-environment).

# UNIT 5: SOFTWARE PROJECT MANAGEMENT

## 5.1 Project Manager Responsibilities & Planning

**Responsibilities:** Scope definition, resource management, risk mitigation, scheduling, quality assurance, budgeting, team leadership, and stakeholder communication.

## 5.2 Size & Cost Estimation Techniques

**LOC-Based Estimation:** Direct size measurement. *Effort = Total LOC / Productivity*. *Cost = Effort × Cost per Person-Month*.

**Function Point (FP) Analysis:** Indirect size measurement based on functionality delivered.

| FP Component | Description |
|---|---|
| External Inputs (EIs) | Data originating from user / external app entering the system. |
| External Outputs (EOs) | Derived report/screen generated for the user. |
| External Inquiries (EQs) | Online input resulting in immediate simple output retrieval. |
| Internal Logical Files (ILFs) | Logical data grouping maintained inside system boundary. |
| External Interface Files (EIFs) | Data referenced by system but maintained externally. |

**FP Formula:** `FP = Count_Total × [0.65 + 0.01 × ∑(Fi)]` where Fi are 14 Value Adjustment Factors (VAF).

**3-Point Estimation (PERT Sizing):** `Expected Value (EV) = (Optimistic + 4 × Most_Likely + Pessimistic) / 6`

## 5.3 COCOMO Cost Model (Boehm)

**Constructive Cost Model (COCOMO):** Predicts effort and duration based on KLOC.

| Project Mode | Complexity & Team Experience | a1 | a2 | b1 | b2 |
|---|---|---|---|---|---|
| Organic | Small, experienced team, well-understood stable environment. | 2.4 | 1.05 | 2.5 | 0.38 |
| Semidetached | Medium size, mixed experience, rigid/unrigid requirements. | 3.0 | 1.12 | 2.5 | 0.35 |
| Embedded | Complex, tight constraints, hardware/software coupling. | 3.6 | 1.20 | 2.5 | 0.32 |

**COCOMO Equations:** `Effort = a1 × (KLOC)^a2 Person-Months` | `Tdev = b1 × (Effort)^b2 Months`.

## 5.4 Scheduling & Effort Distribution

**40-20-40 Rule for Effort Allocation:** 40% Analysis & Design, 20% Coding, 40% Verification & Testing.

**Scheduling Tools:** PERT & CPM (identify Critical Path — sequence determining minimum project duration), Gantt Charts (bar charts mapping tasks against calendar timelines).

## 5.5 Risk Management & RMMM Plan

**Risk Definition:** A potential future problem characterized by *Uncertainty* and *Loss*.

**Risk Categories:** Project Risks (schedule/budget), Technical Risks (quality/design), Business Risks (Market, Strategic, Sales, Management, Budget).

**4 Steps in Risk Management:** (1) Identification, (2) Projection/Analysis (Probability & Impact), (3) Ranking, (4) Contingency Planning.

**RMMM Plan: R**isk **M**itigation (avoidance), **M**onitoring (tracking), and **M**anagement (contingency execution documented via *Risk Information Sheets - RIS*).

# FINAL REVISION & EXAM PREPARATION SECTION

## 8.1 One-Page Quick Revision Formula Sheet

| Formula Name | Mathematical Equation / Expression | Key Variables |
|---|---|---|
| PERT 3-Point Estimate | EV = (O + 4M + P) / 6 | O = Optimistic, M = Most Likely, P = Pessimistic |
| LOC Effort | Effort = Total LOC / Productivity | Effort in Person-Months (PM) |
| Function Points (FP) | FP = Count_Total × [0.65 + 0.01 × ∑Fi] | Fi = 14 Complexity Adjustment Factors (0-5) |
| COCOMO Effort | Effort = a1 × (KLOC)^a2 [PM] | a1, a2 constants (Organic: 2.4, 1.05) |
| COCOMO Schedule | Tdev = b1 × (Effort)^b2 [Months] | b1, b2 constants (Organic: 2.5, 0.38) |
| Cyclomatic Complexity | V(G) = E - N + 2P OR V(G) = P_pred + 1 | E = Edges, N = Nodes, P_pred = Predicate Nodes |

## 8.2 Important Syllabus Definitions Glossary

- **Software Engineering:** Systematic, disciplined, quantifiable approach to software development, operation, and maintenance.
- **Software Requirements Specification (SRS):** Official document describing what a software system should do before development.
- **Coupling:** The degree of interdependence between different software modules (Goal: LOW).
- **Cohesion:** The degree of internal functional strength/relationship within a single module (Goal: HIGH).
- **Verification:** Static evaluation checking if software matches specifications ('Building the product right').
- **Validation:** Dynamic evaluation checking if software satisfies user needs ('Building the right product').
- **White-Box Testing:** Structural testing based on internal code logic and execution paths.
- **Black-Box Testing:** Behavioral testing based on functional requirements without internal code knowledge.
- **RMMM Plan:** Risk Mitigation, Monitoring, and Management plan outlining proactive risk strategies.

## 8.3 Master Syllabus Comparisons Table

| Comparison Pair | Key Difference 1 | Key Difference 2 |
|---|---|---|
| Waterfall vs Agile | Waterfall is linear, sequential, rigid. | Agile is iterative, flexible, sprint-based. |
| FR vs NFR | FR defines WHAT system does. | NFR defines HOW WELL it performs. |
| Verification vs Validation | Static (reviews/inspections, QA). | Dynamic (executing code, Test team/Users). |
| White Box vs Black Box | Internal logic/code path testing. | External inputs/outputs functional testing. |
| Walkthrough vs Inspection | Informal peer review session. | Formal 3-8 member inspection team with checklist. |
| Driver vs Stub | Dummy main calling module (Bottom-Up). | Dummy subprogram called by module (Top-Down). |
| LOC vs Function Points | Direct metric based on code lines. | Indirect metric based on features delivered. |

## 8.4 20 Practice Exam Questions with Answer Key

**Q1: Explain Brooks's Law in the context of software management myths.**
Ans: Brooks's Law states that 'adding manpower to a late software project makes it later' due to training time and exponential increase in communication overhead.

**Q2: Differentiate between Software Verification and Validation.**
Ans: Verification ensures 'building the product right' via static reviews without running code. Validation ensures 'building the right product' via dynamic execution against user requirements.

**Q3: What are the 4 values of the Agile Manifesto?**
Ans: (1) Individuals & Interactions over processes/tools; (2) Working Software over comprehensive documentation; (3) Customer Collaboration over contract negotiation; (4) Responding to Change over following a plan.

**Q4: Define High Cohesion and Low Coupling. Why are they desirable?**
Ans: High cohesion means a module focuses on a single, well-defined task. Low coupling means minimal interdependence between modules. Together, they maximize maintainability, reusability, and testability.

**Q5: Calculate EV for Optimistic=4, Most Likely=7, Pessimistic=16.**
Ans: EV = (4 + 4(7) + 16) / 6 = (4 + 28 + 16) / 6 = 48 / 6 = 8 Person-Months.

**Q6: What is the 40-20-40 rule in project scheduling?**
Ans: 40% of overall effort is allocated to Analysis and Design, 20% to Coding, and 40% to Verification and Testing.

**Q7: Differentiate between Drivers and Stubs in Unit Testing.**
Ans: A Driver simulates a main calling program used in Bottom-Up testing. A Stub simulates a lower-level called module used in Top-Down testing.

**Q8: List the 5 generic activities of the Software Process Framework.**
Ans: Communication, Planning, Modeling, Construction, and Deployment.

**Q9: What is the difference between Alpha Testing and Beta Testing?**
Ans: Alpha testing is performed at developer site by users in a controlled setup. Beta testing is performed at customer live site in an uncontrolled real-world setup.

**Q10: Calculate Effort for Organic mode with 32 KLOC using Basic COCOMO (a1=2.4, a2=1.05).**
Ans: Effort = 2.4 * (32)^1.05 = 2.4 * 37.89 = 91 Person-Months.

**Q11: What are the 3 Golden Rules of User Interface Design?**
Ans: (1) Place the user in control; (2) Reduce the user's memory load; (3) Make the interface consistent.

**Q12: Describe the 3-Tier Architecture layers.**
Ans: (1) Presentation Layer (UI); (2) Business Logic Layer (Services/Rules); (3) Data Access Layer (Database).

**Q13: Name 5 Non-Functional Requirements (NFRs).**
Ans: Availability, Reliability, Scalability, Security, Maintainability.

**Q14: Differentiate Code Walkthrough from Code Inspection.**
Ans: Walkthrough is an informal peer simulation. Inspection is a formal audit by a 3-8 person team using standardized checklists.

**Q15: What are the 5 components of Function Point analysis?**
Ans: External Inputs (EIs), External Outputs (EOs), External Inquiries (EQs), Internal Logical Files (ILFs), External Interface Files (EIFs).

**Q16: Explain Smoke Testing.**
Ans: Smoke testing verifies build stability by testing critical pathways before undertaking deeper regression/system testing.

**Q17: What are the 7 phases of Requirements Engineering?**
Ans: Inception, Elicitation, Elaboration, Negotiation, Specification, Validation, Management.

**Q18: Define Basis Path Testing.**
Ans: A white-box testing technique calculating cyclomatic complexity to ensure every independent execution path is tested at least once.

**Q19: What is RMMM in Risk Management?**
Ans: Risk Mitigation (avoidance), Risk Monitoring (tracking), and Risk Management (contingency execution).

**Q20: Why is software considered to 'deteriorate' rather than 'wear out'?**
Ans: Software doesn't degrade physically; rather, frequent changes introduce unexpected side effects that increase defect density over time.